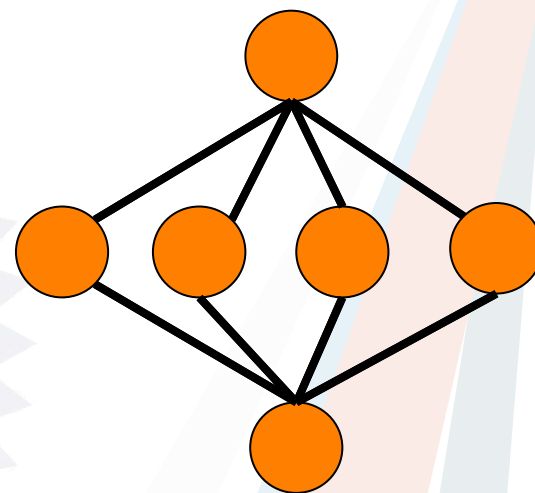
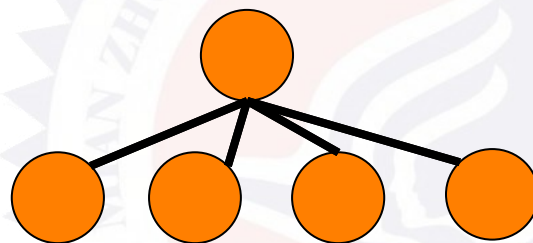


# 树形数据结构

主讲：黄凤林

# 数据结构

## 维护数据与数据之间的关系的存储结构



线性类数据结构:

数组: 任意位置访问存储

栈: 一端操作 (查询、插入、删除等)

队列: 两端操作 (队尾进、队首出)

链表: 每次查询从头开始, 插入和删除比较方便

# 树形数据结构

- **数据结构除了线性以外，其他的节本都是树形的。**
- 树形的数据结构众多，基本上都是用完全二叉树的形态来构建的，也就是如果**节点编号为 $i$** ，则其**左孩子编号为 $2*i$** ，**右孩子为 $2*i+1$** ，其父节点编号为 $i/2$ ；
- 因此，我们就可以利用数组来模拟构建树形数据结构
- 常见的树形数据结构有：
- 堆（优先队列）、并查集、树状数组、线段树、字典树

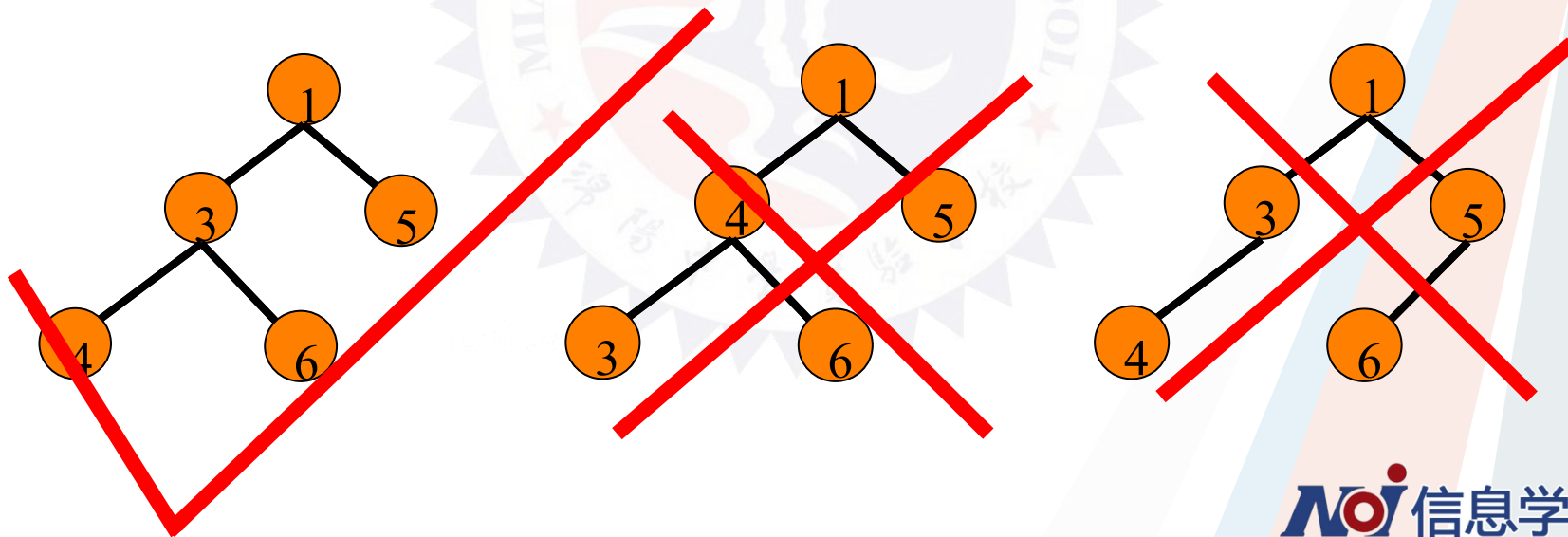
# 堆(优先队列)

**堆:**

堆其实就是一棵完全二叉树（若设二叉树的深度为 $h$ ，除第 $h$ 层外，其它各层 $(1 \sim h-1)$ 的结点数都是两个，第 $h$ 层所有的结点都连续集中在最左边），且满足以下两点：

- 1、根节点元素值小于（大于）左孩子和右孩子的值。
- 2、左右子树又满足第1个性质。

这就是堆（优先队列的实现原型），如下图所示。





## ➔ 堆的应用

### ➤ 堆得用途：

常常用于在一个序列N中，存在**多次插入，多次取值**这样的情况。

### ➤ 堆得操作过程（与优先队列一样）：

**插入：**向堆中插入元素（建堆）

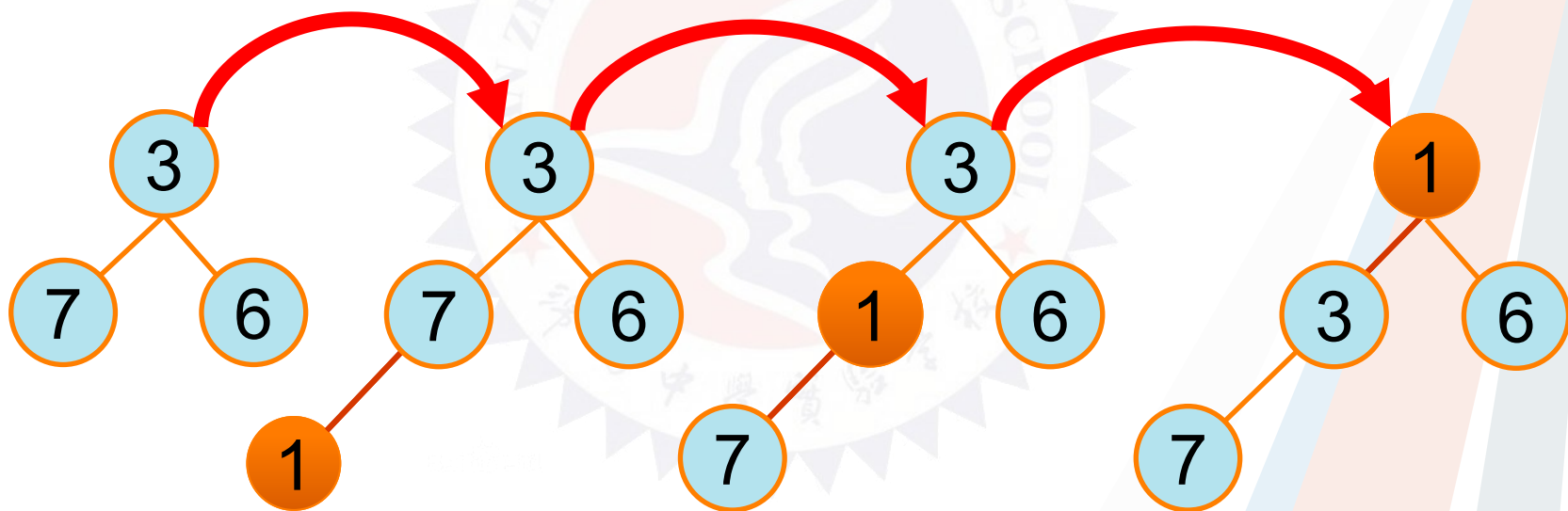
**取出：**取出堆中最优元素（堆顶元素）

## 堆的操作—插入操作

### ➤ 插入元素，一共两步：

- 1、把新加入元素，放在堆尾。
- 2、从堆尾向上调整，直到不能调整或到达堆顶为止。

以小根堆为例，如下图：插入元素1



元素1放在堆尾

元素1向上调整

元素1继续向上调整

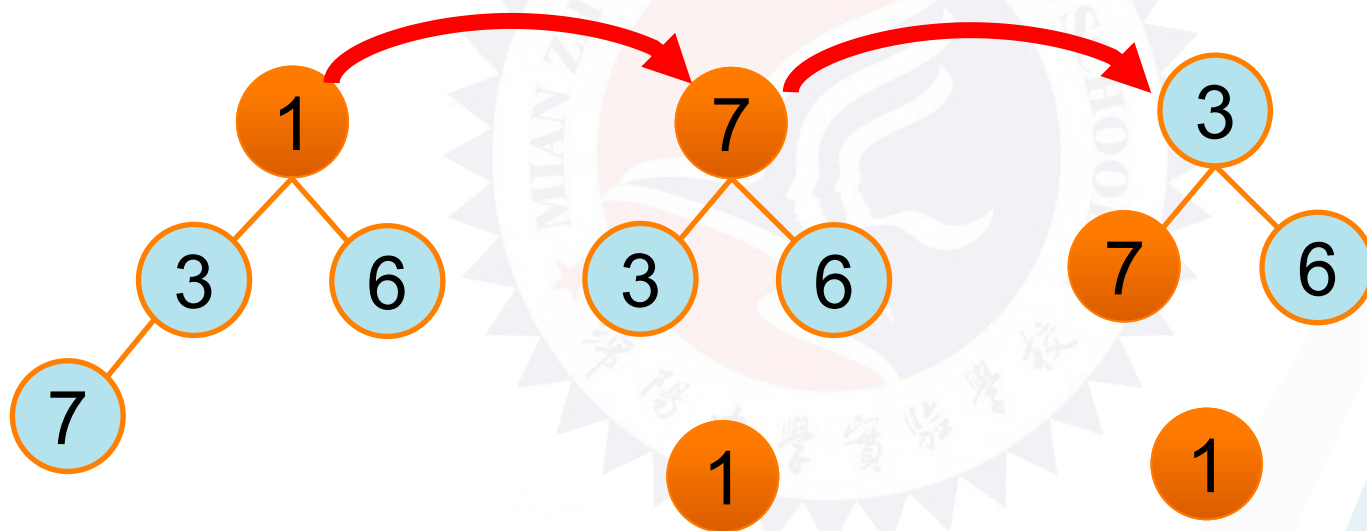
## ➔ 插入操作—实现代码

```
inline void insert(int x){  
    //插入元素x，第一步把元素x放在堆尾  
    heap[++cnt]=x;  
    //第二步，向上调整  
    int fa=cnt;  
    while(fa>1 && heap[fa]<heap[fa>>1]){  
        swap(heap[fa],heap[fa>>1]);  
        fa=fa>>1;  
    }  
}
```

# 堆的操作---取出操作

## ➤ 取出元素，一共三步

- 1、取出堆顶元素
- 2、把堆尾元素放到堆顶
- 3、从堆顶元素开始向下调整，直到不能调整或到达堆尾



取出元素1  
把堆尾元素7放在堆首

从堆首7向下调整





## 取出操作—实现代码

```
inline int get(){
    int h=heap[1];//取出堆首元素
    heap[1]=heap[cnt--];//把堆尾元素放到堆首
    int chi=1;
    //向下调整
    while(chi*2+1<=cnt && heap[chi]>min(heap[chi*2],heap[chi*2+1])){
        if(heap[chi*2]<=heap[chi*2+1]){
            swap(heap[chi],heap[chi*2]);
            chi=chi*2;
        } else {
            swap(heap[chi],heap[chi*2+1]);
            chi=chi*2+1;
        }
    }
    if(chi*2<cnt && heap[chi]>heap[chi*2]){//只有左孩子，千万别搞忘记
        swap(heap[chi],heap[chi*2]);
    }
    return h;
}
```



# 例1. 果子合并

## ➤ 果子合并

在一个果园里，多多已经将所有的果子打了下来，而且按果子的不同种类分成了不同的堆。多多决定把所有的果子合成一堆。

每一次合并，多多可以把两堆果子合并到一起，消耗的体力等于两堆果子的重量之和。可以看出，所有的果子经过 $n-1$ 次合并之后，就只剩下一堆了。多多在合并果子时总共消耗的体力等于每次合并所耗体力之和。

因为还要花大力气把这些果子搬回家，所以多多在合并果子时要尽可能地节省体力。假定每个果子重量都为1，并且已知果子的种类数和每种果子的数目，你的任务是设计出合并的次序方案，使多多耗费的体力最少，并输出这个最小的体力耗费值。

例如有3种果子，数目依次为1，2，9。可以先将1、2堆合并，新堆数目为3，耗费体力为3。接着，将新堆与原先的第三堆合并，又得到新的堆，数目为12，耗费体力为12。所以多多总共耗费体力 $=3+12=15$ 。可以证明15为最小的体力耗费值。

果子堆数 $\leq 100000$

## ➔ 问题分析

- 很明显，这道题是一道贪心的题目，可以证明，每次取最小的两堆合并会使得最后的体力值最小。

那么，这道题的问题就在于怎么找最小的两堆果子了。

注意到，每次合并完果子会删掉两堆，并添加一堆新的，如果采用线性表，时间复杂度将高达 $O(n^2)$ ，对于 $n \leq 100000$ 是不够的。

- **所以，我们考虑用小根堆优化，算法：**

- 插入：每读入一个数插入堆中， $n$ 个数插入完成后
- 合并：每次取两个堆顶元素合并，并插入合并后的数，总共合并 $n-1$ 次。



## ➔ 例2. 序列和的前n小元素

### ➤ 序列和的前n小元素

给出两个长度为n的有序表A和B，在A和B中各任取一个，可以得到 $n^2$ 个和，求这些和最小的n个。

➤ 数据范围：  $N \leq 100000$



## 例3. P1168 中位数

### ➤ 题目描述

给出一个长度为 $N$ 的非负整数序列 $A_i$ ，对于所有 $1 \leq k \leq (N+1)/2$ ，输出 $A_1, A_3, \dots, A_{2k-1}$ 的中位数。即前 $1, 3, 5, \dots$ 个数的中位数。

### ➤ 说明/提示

对于20%的数据， $N \leq 100$ ；

对于40%的数据， $N \leq 3000$ ；

对于100%的数据， $N \leq 100000$ 。

➤ **中位数：**如果总数个数是奇数的话，按从小到大顺序，取中间那个数。

## ➔ 中位数—问题分析

- 可以考虑线段树来求解区间的中位数，但是比较麻烦，不太好找。我们可以考虑(堆)优先队列

**开两个堆，一个小根堆，一个大根堆，始终保证大根堆元素的值 $\leq$ 小根堆元素的值，并且大根堆的数量=小根堆+1**

那么这样我们就可以发现，**大根堆堆顶的数就是中位数**，因为如果一共有 $(2n+1)$ 个数。大根堆有 $n+1$ 个数，除去堆顶的元素还有 $n$ 个，小根堆有 $n$ 个数，并且满足大根堆 $\leq$ 小根堆，那么该数就是中位数

每次新插入一个数，只需要维护两个堆的性质不变就可以了，只需要将堆顶元素和该数比较然后出入队就可以了

## → 优先队列 (STL堆)

➤ 头文件: `#include <queue>`

➤ 声明方式:

1、普通方法:

`priority_queue<int> q;` //按照元素从大到小的顺序出队

`priority_queue<int,vector<int>,greater<int> > q;` //按照元素从小到大的顺序出队

2、结构体声明方式

```
struct node {  
    int x, y;  
    friend bool operator<(node a, node b){  
        return a.x > b.x; //结构体中, x小的优先级高  
    }  
};  
priority_queue<node>q; //定义方法
```





# → 优先队列

## 二、priority\_queue

基本操作：

`empty()`          如果队列为空，则返回真

`pop()`            删除对顶元素，删除第一个元素

`push()`           加入一个元素

`size()`            返回优先队列中拥有的元素个数

`top()`            返回优先队列对顶元素，返回优先队列中有最高优先级的元素

在默认的优先队列中，优先级高的先出队。在默认的int型中先出队的为较大的数。





# 中位数—实现代码

```
struct node1{
    int x;
    node1(int xx) {x=xx;}
    friend bool operator<(node1 a, node1 b){
        return a.x>b.x;//x小的优先级高
    }
    /*bool operator < (const node1 &a) const {
        return x>a.x;//x小的优先级高
    }*/
};

struct node2{
    int x;
    node2(int xx){x=xx;}
    friend bool operator<(node2 a,node2 b){
        return a.x<b.x;//x大的优先级高
    }
    /*bool operator < (const node2 &a) const {
        return x<a.x;//x大的优先级高
    }*/
};

priority_queue<node1> minq;//小根堆
priority_queue<node2> maxq;//大根堆,默认的
```

```
n=(n+1)/2;
printf("%d ",a[1]);
maxq.push(node2(a[1]));
for(int i=1;i<=n-1;i++){
    //第一步: 取一个元素加入小根堆
    if(a[i*2]<maxq.top().x){
        minq.push(node1(maxq.top().x));
        maxq.pop(); maxq.push(node2(a[i*2]));
    } else minq.push(node1(a[i*2]));
    //第二步: 取第二个元素加入大根堆
    if(a[i*2+1]>minq.top().x){
        maxq.push(node2(minq.top().x));
        minq.pop(); minq.push(node1(a[i*2+1]));
    } else maxq.push(node2(a[i*2+1]));
    //第三步: 输出大根堆堆顶元素
    printf("%d ",maxq.top().x);
}
```

## 例4. 丑数

### ➤ 丑数

素因子都在集合{2, 3, 5, 7}的数称为ugly number  
求第n大的丑数( $n \leq 100000$ )

### ➤ 问题分析:

初始: 把1放入优先队列中;

每次从优先队列中取出一个元素k, 把 $2k$ ,  $3k$ ,  $5k$ ,  $7k$ 放入优先队列中;

从2开始, 取出的第n个元素就是第n大丑数;

每取出一个数, 插入4个数, 因此任何堆里的元素是 $O(n)$ 的, 时间复杂度为 $O(n \cdot \log n)$

➤ 思考: 如果集合元素个数m与n同阶, 时间复杂度将变为怎样? 如何优化?

## 例5. 轮廓线

### ➤ 轮廓线

每一个建筑物用一个三元组表示(L, H, R), 表示  
左边界, 高度和右边界

轮廓线用X, Y, X, Y...这样的交替式表示

下图的轮廓线为: (1, 11, 3, 13, 9, 0, 12, 7, 16, 3, 19, 18,  
22, 3, 23, 13, 29, 0)

给N个建筑, 求轮廓线

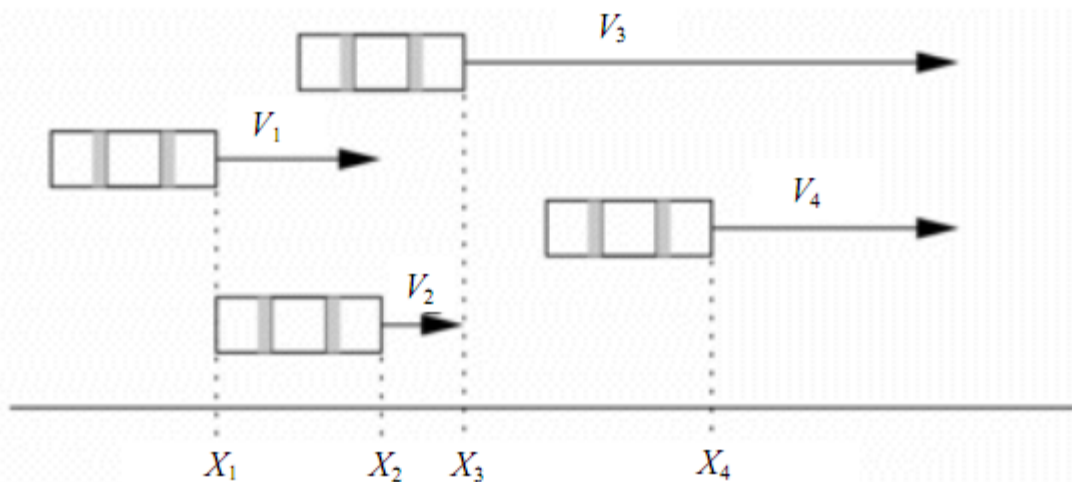
## 例6. 赛车

### ➤ 赛车

有 $n$ 辆赛车从各不相同的地方以各种的速度(速度 $0 < v_i < 100$ )开始往右行驶，不断有超车现象发生(如下图)。

给出 $n$ 辆赛车的描述(位置 $x_i$ ，速度 $v_i$ )，赛车已按照位置排序( $x_1 < x_2 < \dots < x_n$ )

输出超车总数以及按时间顺序的前 $m$ 个超车事件



## → 赛车—问题分析

- 事件个数 $O(n^2)$ , 因此只能一个一个求
- 给定两辆车, 超越时刻预先可算出
- **第一次超车可能在哪些辆车之间?**
- **维护所有车的前方相邻车和追上时刻**
  - 局部: 此时刻不一定是该车下个超车时刻!
  - 全局: 所有时刻的最小值就是下次真实超车时刻
- **维护: 超车以后有什么变化?**
  - 相对顺序变化...改变三个车的前方相邻车
  - 重新算追上时刻, 调整三个权
  - 简单的处理方法: 删除三个再插入三个

## 例7. 可怜的奶牛

### ➤ 可怜的奶牛

**农夫John**有 $n$  ( $n \leq 100\ 000$ ) 头奶牛，可是由于它们产的奶太少，农夫对它们很不满意，决定每天把产奶最少的一头做成牛肉干吃掉。但还是有一点舍不得，John打算如果不止有一头奶牛产奶最少，当天就大发慈悲，放过所有的牛。

由于John的奶牛产奶是周期性的，John在一开始就能可以了解所有牛的最终命运，不过他的数学很差，所以请你帮帮忙，算算最后有多少头奶牛可以幸免于难。每头奶牛的产奶周期 $T_i$ 可能不同，但不会超过10。在每个周期中，奶牛每天产奶量不超过200。

## → 可怜的奶牛—问题分析

如果采用最笨的方法，每次先求出每头牛的产奶量，再求最小值，则每天的复杂度为 $O(n)$ ，总复杂度为 $O(Tn)$ ，其中 $T$ 是模拟的总天数。由于周期不超过10，如果有的牛永远也不会被吃掉，那么我们需要多模拟2520天（1, 2, 3, ..., 10的最小公倍数）才能确定；

周期同为 $t$ 的奶牛在没有都被吃掉之前，每天的最小产奶量也是以 $t$ 为周期的。因此如果把周期相同的奶牛合并起来，每天只需要比较10类奶牛中每类牛的最小产奶量就可以了，每天的复杂度为 $O(k)$ ，其中 $k$ 为最长周期。



## ➔ 可怜的奶牛—问题分析

- 假设周期为6的牛有4头，每次只需要比较k组牛的“代表”就可以了，每天模拟的时间复杂度为 $O(k)$ 。

| 项 目  | 第 $6n+1$ 天 | 第 $6n+2$ 天 | 第 $6n+3$ 天 | 第 $6n+4$ 天 | 第 $6n+5$ 天 | 第 $6n+6$ 天 |
|------|------------|------------|------------|------------|------------|------------|
| 牛1   | 2          | 5          | 3          | 5          | 7          | 4          |
| 牛2   | 3          | 1          | 6          | 7          | 5          | 4          |
| 牛3   | 5          | 3          | 3          | 5          | 3          | 9          |
| 牛4   | 4          | 4          | 3          | 8          | 8          | 2          |
| 合并结果 | 2（牛1）      | 1（牛2）      | 3（多）       | 5（多）       | 3（牛3）      | 2（牛4）      |



## ➡ 可怜的奶牛—问题分析

➤ 只要周期为6的牛都不被吃掉，这个表一直是有效的。但是在吃掉一头奶牛后，我们需要修改这个表，使它仍然记录着每天的最小产奶量：

方法一：重新计算，时间 $O(h)$ ，其中 $h$ 是该组的牛数

方法二：把一个周期中每天的最小产奶量组织成堆，每次删除操作的复杂度是 $O(k\log h)$

➤ 由于每头奶牛最多被吃掉一次，因此用在维护“最小产奶量结构”的总复杂度不超过 $O(nk\log n)$ 。

➤ 每天复杂度为 $O(k)$ ，总复杂度为 $O(Tk+nk\log n)$



## 例8. 黑匣子

### ➤ 黑匣子

我们使用黑匣子的一个简单模型。它能存放一个整数序列和一个特别的变量 $i$ 。在初始时刻，黑匣子为空且 $i$ 等于0。这个黑匣子执行一序列的命令。有两类命令：

- ADD( $x$ )：把元素 $x$ 放入黑匣子；
- GET： $i$ 增1的同时，输出黑匣子内所有整数中第 $i$ 小的数。牢记第 $i$ 小的数是当黑匣子中的元素以非降序排序后位于第 $i$ 位的元素。



# 黑匣子—问题分析

## 例7. 黑匣子

| 编号 | 命令         | i | 黑匣子内容                         | 输出 |
|----|------------|---|-------------------------------|----|
| 1  | ADD(3)     | 0 | 3                             |    |
| 2  | GET        | 1 | <b>3</b>                      | 3  |
| 3  | ADD(1)     | 1 | 1, 3                          |    |
| 4  | GET        | 2 | 1, <b>3</b>                   | 3  |
| 5  | ADD(-4)    | 2 | -4, 1, 3                      |    |
| 6  | ADD(2)     | 2 | -4, 1, 2, 3                   |    |
| 7  | ADD(8)     | 2 | -4, 1, 2, 3, 8                |    |
| 8  | ADD(-1000) | 2 | -1000, -4, 1, 2, 3, 8         |    |
| 9  | GET        | 3 | -1000, -4, <b>1</b> , 2, 3, 8 | 1  |
| 10 | GET        | 4 | -1000, -4, 1, <b>2</b> , 3, 8 | 2  |
| 11 | ADD(2)     | 4 | -1000, -4, 1, 2, 2, 3, 8      |    |



## ➔ 黑匣子—问题分析

- 做法与中位数那道题类似，因此，大家自己想办法解决



# 堆优化dijkstra

## ➤ 最小序列问题

给定一个 $N \times N$  ( $N \leq 100$ ) 的正整数矩阵。需要在矩阵中寻找一条从起始位置到终止位置的路径(可沿上下左右四个方向), 并且要求路径中经过的所有数字, 其相邻数字之差的绝对值之和最小。



## ➔ 问题分析

- 这道题的基本算法很简单，只要用Dijkstra算法求出从起始位置到终止位置的最短路径即可。
- 但这当中存在一个很大的问题： $N \leq 100$ 。这就是说图中点的数目可能多达10000个。
- 此时复杂度为 $O(n^2)$ 的Dijkstra算法就会超时。
- 因此，你可以用spfa，或者dijkstra+堆优化



## → 并查集

- **并查集**是一种树型的数据结构，用于处理一些不相交集合 (Disjoint Sets) 的合并及查询问题。常常在使用中以 father 数组来表示；用于维护集合关系的一种数据结构。
- **并查集用途：**
  - ① 维护一个无向图的连通性，判断  $n$  个点  $m$  条边时最少加多少边可以连通所有点。
  - ② 判断在一个无向图中，两点间加边是否会产生环（最小生成树克鲁斯卡尔中有用到）





## → 并查集

➤ 并查集的操作：共有三种

1 集合的建立

2 集合的元素的查询

3 集合的合并

➤ 并查集的建立，我们可以用一个father数组来模拟， $\text{father}[i]=j$ ，表示元素 $i$ 所在的集合为 $j$ 。

➤ 初始化集合:  $\text{father}[]$  ---  $\text{father}[i]=i$

➤  $\text{father}[i]=i$ 也表示最顶层集合



## → 并查集实现

**集合建立：** father[ ],    **初始化：** father[i]=i

### ➤ 集合元素的查询

```
int find(int x){  
    if(father[x]==x)  
        return x;  
    else  
        return find(father[x]);  
}
```

### ➤ 集合的合并

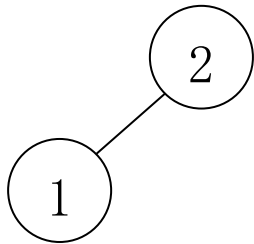
```
void merge(int x,int y){  
    int fx=find(x);  
    int fy=find(y);  
    if(fx!=fy){  
        father[fx]=fy;  
    }  
}
```



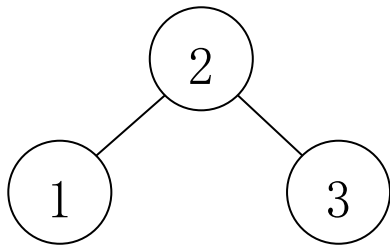
# 并查集分析优化



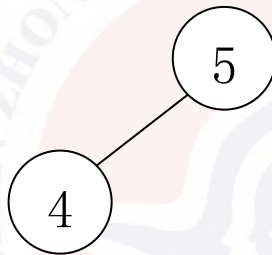
1.合并1和2



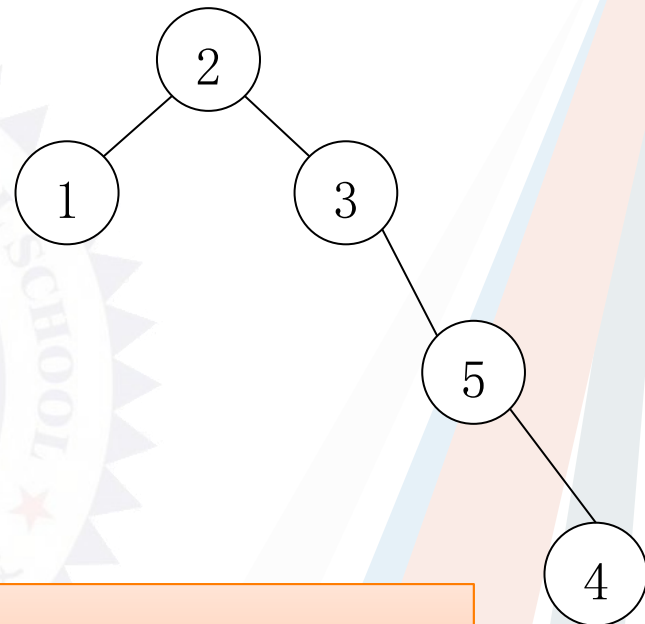
2.合并2和3



3.合并5和4



4.合并5和3



弊端!!!

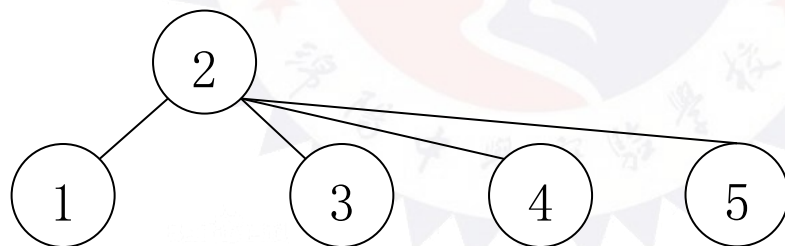
查询4所在的集合时间复杂度很高  
如何优化呢?

## 并查集--路径压缩

上述做法是指就是将元素的父亲结点指来指去的在指，当这棵树是链的时候，可见判断两个元素是否属于同一集合需要 $O(n)$ 的时间，于是路径压缩产生了作用。

**路径压缩**实际上是在找完根结点之后，在递归回来的时候顺便把路径上元素的父亲指针都指向根结点。

这就是说，我们在“合并5和3”的时候，不是简单地将5的父亲指向3，而是直接指向根节点1，如图





## ➔ 路径压缩实现

- 元素所在集合的查询(回溯时做一个记录):

```
int find(int x)
{
    if(father[x] == x)
    {
        return x;
    }
    father[x] = find(father[x]);
    return father[x];
}
```

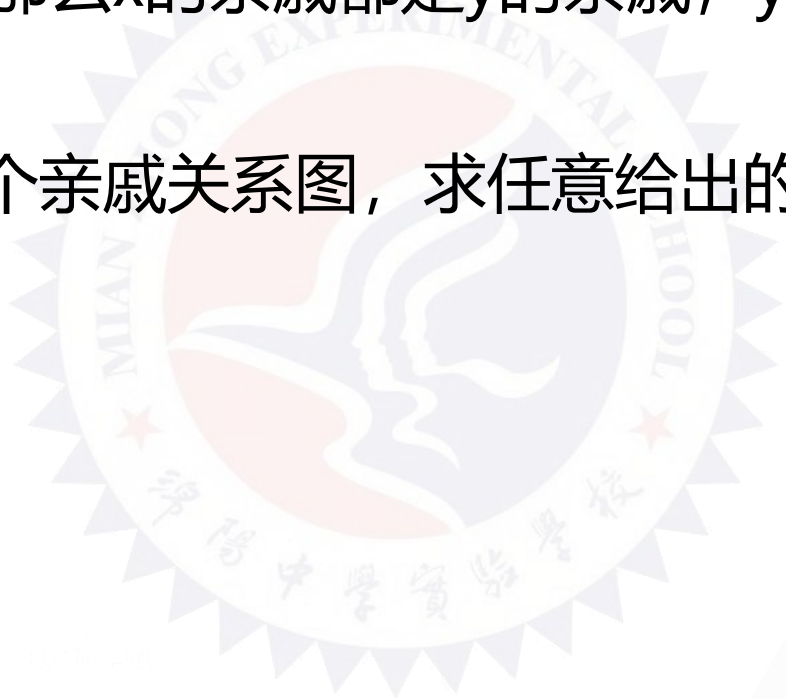


## ➔ 例1. 亲戚(P1551)

### ➤ 题目描述

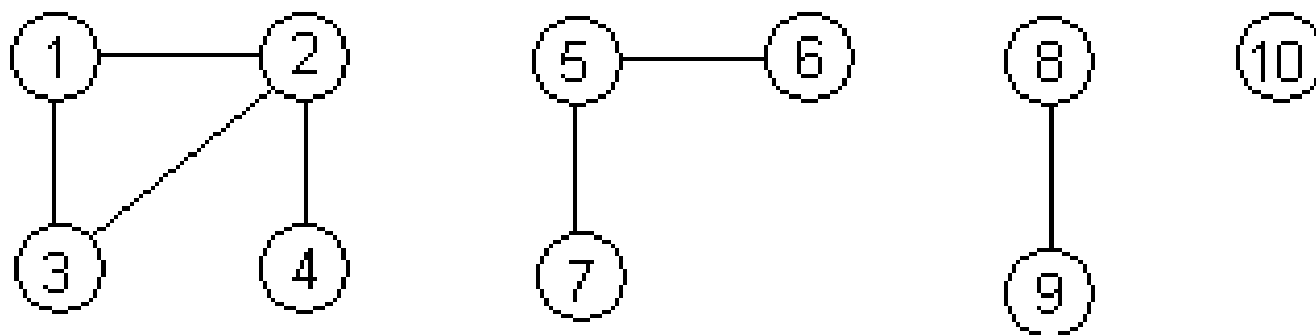
规定： $x$ 和 $y$ 是亲戚， $y$ 和 $z$ 是亲戚，那么 $x$ 和 $z$ 也是亲戚。如果 $x,y$ 是亲戚，那么 $x$ 的亲戚都是 $y$ 的亲戚， $y$ 的亲戚也都是 $x$ 的亲戚。

现在给出某个亲戚关系图，求任意给出的两个人是否具有亲戚关系。



## ➔ 问题分析

- 本质: 是否在图的同一个连通块中
- 问题: 图太庞大, 每次还需要遍历
- 解决: 用并查集



## 例2.最小生成树

### ➤ 问题描述:

在一个具有几个顶点的连通图 $G$ 中, 如果存在子图 $G'$  包含 $G$ 中所有顶点和一部分边, 且不形成回路, 则称 $G'$  为图 $G$ 的生成树, 权值和最小的生成树则称为最小生成树。

### ➤ 如何求解图的最小生成树呢?

我们可以采用按边权值贪心的策略。于是就产生了求最小生成树最经典的算法, **Kruskal**算法。





# → Kruskal

➤ 思想：按照边的权值贪心

➤ 算法执行过程：

- 1.按照输入的边建立边集数组，对边集数组按照权值从小到大排序
- 2.把 $n$ 个点放在生成树中，使得他们各种处于不同集合
- 3.向生成树中从最小的边开始添加边。判断该边的两个顶点是否在同一集合中，如果不是则添加。
- 4.重复第3步，直到添加 $n-1$ 条边，则完成最小生成树的建立

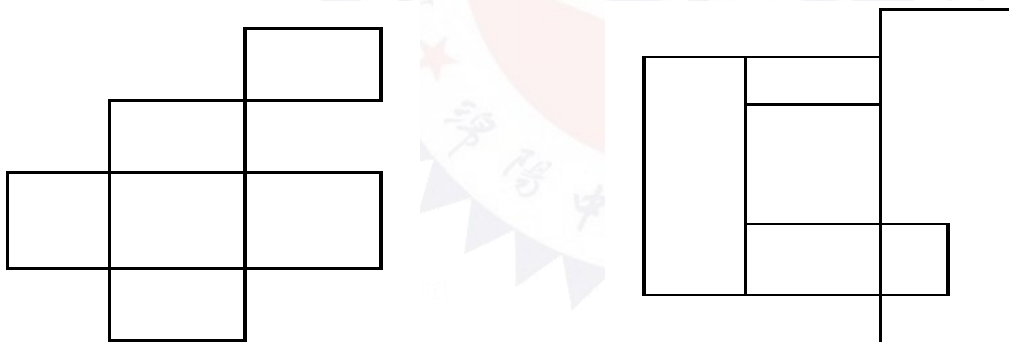
## 例3. 矩形

### 题目描述

在平面上画了N个长方形，每个长方形的边平行于坐标轴并且顶点坐标为整数。我们用以下方式定义印版：

- 每个长方形是一个印版；
- 如果两个印版有公共的边或内部，那么它们组成新的印版，否则这些印版是分离的

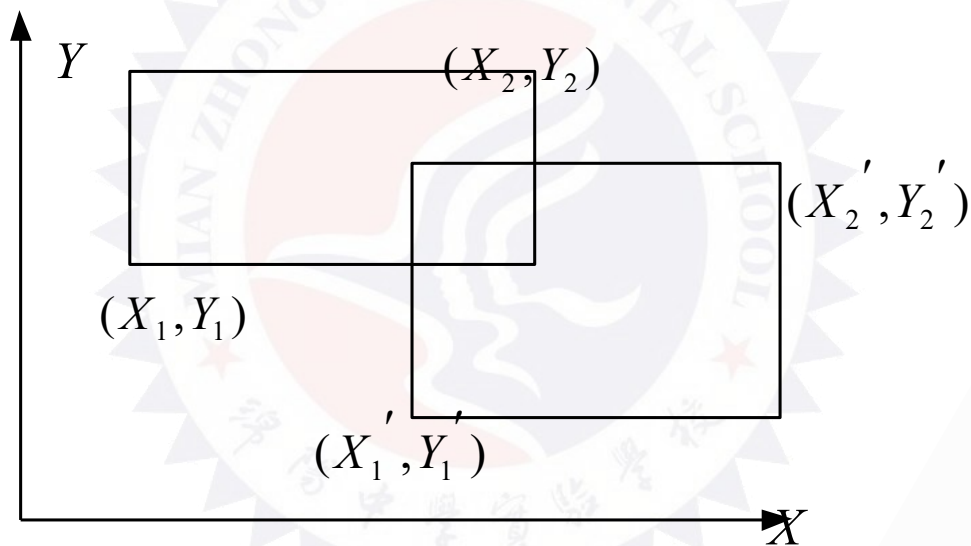
求出印版的个数。左图有两个，右图只有一个





## → 分析

- 把矩形看作点，有公共边的矩形连边，问题转化为求连通分量的个数
- 判断方法：



## → 并查集扩展

并查集一般作用：

- 1、维护节点之间的连通性及求联通快
- 2、动态维护许多具有传递性的关系

并查集实际上是有若干颗树构成的森林，我们可以在树种的每条边上记录一个权值，即维护一个数组 $d$ ，用 $d[x]$ 保存节点 $x$ 到父节点 $fa[x]$ 之间的边权。在每次路径压缩后，每个访问过的节点就会直接指向树根，如果我们同时更新这些节点的 $d$ 值，就可以利用路径压缩过程来统计每个节点到树根之间路径上的一些信息，比如统计集合元素的个数。这就是所谓的“变带权”的并查集。

# → [NOI2002]银河英雄传说

## ➤ 问题描述:

有一个划分成 $N$ 列的星际战场，各列依次编号为 $1, 2, \dots, N$ 。有 $N$ 艘战舰，也依次编号为 $1, 2, \dots, N$ ，其中第 $i$ 号战舰处于第 $i$ 列。有 $M$ 条指令：

1  $M_{ij}$ ：表示让第 $i$ 号战舰所在列的全部战舰保持原有顺序，接在第 $j$ 号战舰所在列的尾部，并且保证第 $i$ 号战舰与第 $j$ 号战舰不在同一列。

2  $C_{ij}$ ：表示询问第 $i$ 号战舰与第 $j$ 号战舰是否处于同一列中，如果在同一列中，他们之间间隔了多少艘战舰

➤  $N \leq 30000, m \leq 5 \times 10^5$

## → 问题分析

利用并查集来处理：

一开始把 $N$ 个战舰看成 $N$ 个独立的集合。在没有路径压缩的情况下， $fa[x]$ 就表示排在第 $x$ 号战舰前面的那个战舰的编号。一个集合的代表(集合的根)就是位于最前面的战舰。另外，让树上每条边带权值为1，这样树上两点之间的距离减1就是二者之间间隔的战舰数量。

在考虑路径压缩的情况下，我们额外建立一个数组 $d$ ， $d[x]$ 记录战舰 $x$ 与 $fa[x]$ 之间的边的权值。在路径压缩把 $x$ 直接指向树根的同时，我们把 $d[x]$ 更新为从 $x$ 到树根的路径上的所有边权和。

## → 问题分析

查询：对d数组的维护

```
int find(int x){  
    if(x==fa[x]) return x;  
    int root=find(fa[x]);  
    d[x]+=d[fa[x]];  
    return fa[x]=root;  
}
```

对于C指令，分别进行find(x)和find(y)完成查询和路径压缩，若二者的返回值相同，这说明x和y处于同一列中。所以d[x]保存了位于x之前的战舰数量，d[y]保存了位于y之前的战舰数量。二者只差减1，就是x和y之间间隔的战舰数量

合并：对d数组和size数组的维护

```
void merge(int x,int y){  
    x=find(x);y=find(y);  
    fa[x]=y;d[x]=size[y];  
    size[y]+=size[x];  
}
```

对于M指令，把x的树根作为y的树根的子节点，连接新边的权值应该为合并之间集合y的大小(依据题意，集合y中全部战舰都排在集合x之前)。因此，我们还需要一个size数组来记录每个树根上集合的大小。



## → 树状数组

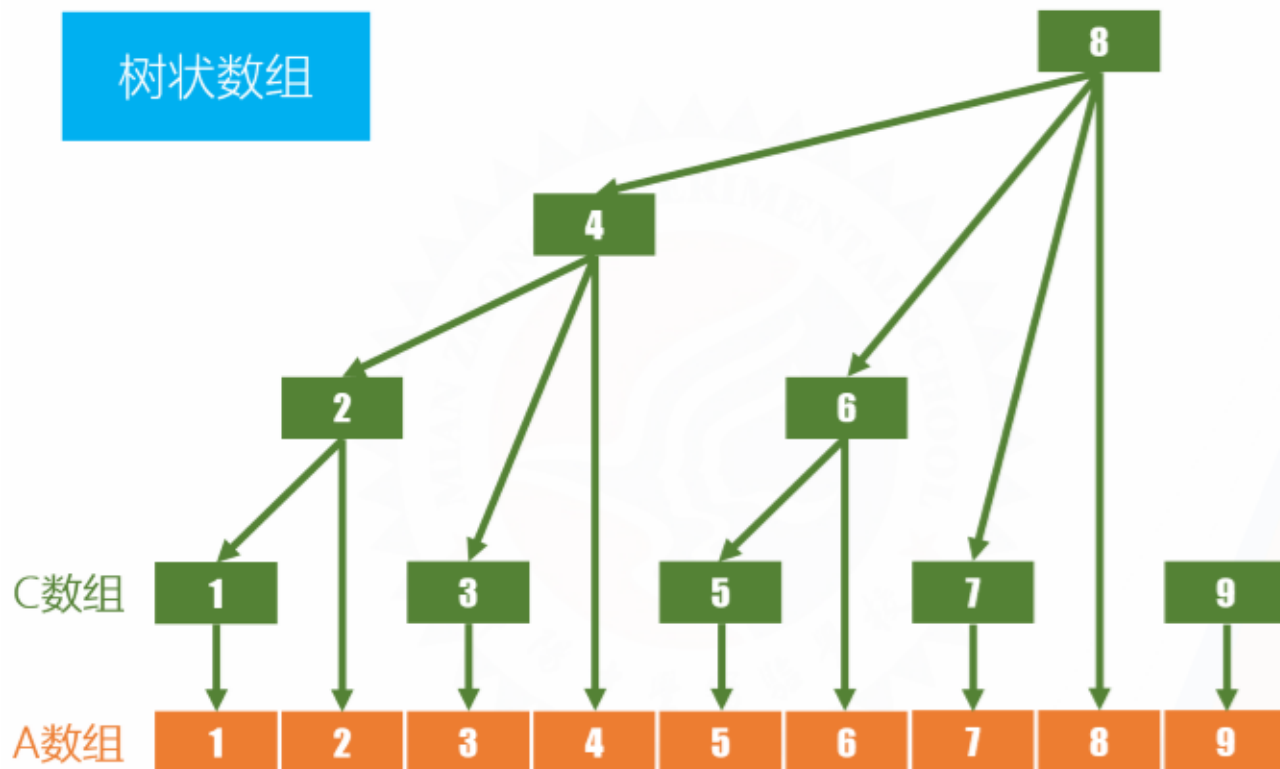
- **树状数组**，顾名思义一种树形的数组，是一种可以将线性结构转化成树状结构的数据结构，从而进行跳跃式扫描，通常使用在高效的**计算数列的前缀和**，**区间和**。同时，我们在运用线段树的时应先考虑是不是可以使用树状数组来解决问题。
- **效率上：树状数组 > 线段树 > 分块**
- **解决问题及扩展上：分块 > 线段树 > 树状数组**
- **树状数组解决问题：**
  - 1.单点更新、单点查询**
  - 2.单点更新、区间查询**
  - 3.区间更新、单点查询**
  - 4.区间更新、区间查询**





# 神秘的树状数组

树状数组



A[]数组:序列给定的原始数组

C[]数组:根据原序列建的树状数组, 用于统计查询, 更新等。

## 神秘的树状数组

### 观察分析:

通过观察发现，每个位置只有一个 $C[i]$ ，如果令每个位置存的就是子节点的值的和，则有

$$C[1] = A[1];$$

$$C[2] = A[1] + A[2];$$

$$C[3] = A[3];$$

$$C[4] = A[1] + A[2] + A[3] + A[4];$$

$$C[5] = A[5];$$

$$C[6] = A[5] + A[6] = C[5] + A[6];$$

$$C[7] = A[7];$$

$$C[8] = A[1] + A[2] + A[3] + A[4] + A[5] + A[6] + A[7] + A[8];$$

## → 神秘的树状数组

- 通过分析发现，**每个C[i]所管辖的范围(统计A[i]的元素个数)各不相同**，那他们有什么规律了？下面看例子：

$5=(101)_2$  末尾连续零的个数为0个，只能管辖1个A[5]

$6=(110)_2$  末尾连续零的个数为1个，能管辖2个A[5,6]

$7=(111)_2$  末尾连续零的个数为0个，只能管辖1个A[7]

$8=(1000)_2$  末尾连续零的个数为3个，能管辖8个A[1,2,...8]

- 于是，我们得到一个什么结论呢？
- 通过例子发现，C[i]管辖的个数与i末尾连续零的个数成指数关系，即管辖个数 **$k=2^j$** ，**j为i在二进制下末尾连续零的个数**



## → 神秘的树状数组

- 进一步分析，这个K有何用呢？（先不管怎么求出K）
- 假如我们利用C数组来求sum[i]的值(sum[i]表示1到i元素的和)。如：i=6，则会发现：**sum[6]=C[6]+C[4]**即可，其中**6与4之间差的值刚好就是k**，那是不是所有都是这个情况呢？

$\text{sum}[5] = \text{C}[5] + \text{C}[4]$ ,  $\text{sum}[7] = \text{C}[7] + \text{C}[6] + \text{C}[4]$ ;

$\text{sum}[8] = \text{C}[8]$ ,  $\text{sum}[9] = \text{C}[9] + \text{C}[8]$ ;

有了这个以后，我们会发现求**sum[i]时，就可以利用C数组实现跳跃式的扫描，加快速度。**

## → 神秘的树状数组

➤ 如何求出  $k=2^j$ ,  $j$  为  $i$  在二进制下末尾连续零的个数。

➤  $K=2^j = i \& (i \wedge (i-1)) = i \& (-i)$  (前辈的智慧)

“&、|、^, >>, <<, ~, ” 为位运算符, 程序中的所有数在计算机内存中都是以二进制的形式储存的。位运算就是直接对整数在内存中的二进制位进行操作。

| C/C++        | P                  |
|--------------|--------------------|
| $a \& b$     | $a \text{ and } b$ |
| $a   b$      | $a \text{ or } b$  |
| $a \wedge b$ | $a \text{ xor } b$ |
| $\sim a$     | $\text{not } a$    |
| $a \ll b$    | $a \text{ shl } b$ |
| $a \gg b$    | $a \text{ shr } b$ |

| 名称   | 符号      | 规则         |
|------|---------|------------|
| 按位与  | &       | 全一则一, 否则为零 |
| 按位或  |         | 有一则一, 否则为零 |
| 按位取反 | ~       | 是零则一, 是一则零 |
| 按位异或 | ^       | 不同则一, 相同则零 |
| 移位   | >> 和 << | 向右、向左移位    |

## → 神秘的树状数组

➤ K的求法,  $K=2^j = i \& (i \wedge (i-1)) = i \& (-i)$

$K = i \& (i \wedge (i-1))$  的求解过程, 如:  $k=6$ , 则  $6 = (110)_2$

$110 \wedge 101 = 10$ ,  $110 \& 10 = 10$ , 也就是2

再如  $k=12$ , 则  $12 = (1100)_2$

$1100 \wedge 1011 = 111$ ,  $1100 \& 0111 = 100$ , 也就是4

$i \& (-i)$  怎么来的呢? 这就要讲一讲数字在计算机内存中的存储规则 (初赛很喜欢考)

# 原码 补码 反码

## 一、数的原码、补码和反码表示

### (1) 机器数与真值

计算机中，表示数值的数字符号只有0和1两个数码，规定最高位为符号位，并用0表示正数符号，用1表示负数符号。这样，机器中的数值和符号全“数码化”了。为简化机器中数据的运算操作，人们采用了原码、补码、反码及移码等几种方法对数值位和符号位统一进行编码，**计算机内部则采用补码进行存储**。为区别起见，我们将数在机器中的这些编码表示称为机器数（如：10000001），而将原来一般书写表示的数称为机器数的真值（如：-0000001）。

### (2) 原码表示法

原码表示法是一种简单的机器数表示法，即符号和数值表示法，设 $x$ 为真值， $[x]$ 原为机器数表示。

例：设  $x=1100110$ ，则  $[x]_{\text{原}}=01100110$ ， $x=-1100111$ ，则  $[x]_{\text{原}}=11100111$

### (3) 反码表示法

**正数的反码就是真值本身；负数的反码，只须对符号位以外各位按位“求反”（0变1，1变0）即可。**

例：设  $x=1100110$ ，则  $[x]_{\text{反}}=01100110$ ， $x=-1100111$ ，则  $[x]_{\text{反}}=10011000$

### (4) 补码表示法

**负数用补码表示时，可以把减法转化成加法。正数的补码就是真值本身；负数的补码是符号位为1，数值各位取反（0变为1、1变为0），最低位加1。**

例：设  $x=1100110$ ，则  $[x]_{\text{补}}=01100110$ ， $x=-1100111$ ，则  $[x]_{\text{补}}=10011001$



## → 神秘的树状数组

➤ K的求法,  $K=2^j = i \& (i \wedge (i-1)) = i \& (-i) =$

$K=i \& (-i)$ , 如:  $k=6$ , 则  $6=(0110)_2$ ,  $(-6)=(1010)_2$

$0110 \& 1010 = 10$ , 也就是2

再如  $k=12$ , 则  $12=(01100)_2$ ,  $(-12)=(10100)_2$

$01100 \& 10100 = 100$ , 也就是4

感受到计算机的强大之处了吧!!! 前辈需要我们膜拜!!!

K的求解问题成功过解决了。

就可以利用k快速遍历求出sum[i]的值。



## → 神秘的树状数组

### ➤ Sum[i]的求解过程

第一步：累加C[i]的值

第二步：向下统计，利用K，快速实现向下跳跃到需要统计的位置，也就是 $i = i - (i \& (-i))$ 。

第三步：重复第二步，直到 $i = 0$ 为止。

时间复杂度： $O(\log n)$

```
inline long long get(int x){  
    long long ans=0;  
    while(x>0){  
        ans+=c[x];  
        x-=(x&-x);  
    }  
    return ans;  
}
```

## → 神秘的树状数组

- 我们再来看修改(如果单纯的统计, 我们利用前缀和就可以轻松解决)。假如我们这里是单点修改(区间修改较难), **将 $a[i]$ 的值加上一个 $\delta$ , 我们就需要向上, 凡是能管辖到 $a[i]$ 的C数组, 全部加上 $\delta$ 的值。**
- 这是为什么呢?
- 假设 $C[j]$ 能管辖到 $C[i]$ , 以后在向下统计时, 我们只需要到 $C[j]$ 就行了, 无需到 $C[i]$ ,  $C[j]$ 管辖不到的范围, 统计时一定会到达相关位置, 我们再累加上即可。
- 如 $a[5]$ 更新, 则仅需更新:  $C[5] \rightarrow C[6] \rightarrow C[8] \rightarrow C[16] \dots$
- 分析发现这里中间差的又是 $K$ , 所以我们又可以利用 $K$ 快速实现向上扫描更新。

## → 神秘的树状数组

### ➤ $a[i]$ 的更新操作过程

第一步：更新 $C[i]$ 的值

第二步：向上更新，利用 $K$ ，快速实现向上跳跃需要到更新的位置，也就是 $i = i + (i \& (-i))$

第三步：重复第二步，直到 $i > n$ 为止。

时间复杂度：  $O(\log n)$

```
inline void update(int x,int val)
{
    while(x<=n){
        c[x]+=val;
        x+=(x&-x);
    }
}
```

## → 树状数组应用

### ➤ 敌兵布阵

敌人有 $N$  ( $N \leq 500000$ ) 个工兵营地，开始时第 $i$ 个工兵营地里有 $a_i$ 个人 ( $1 \leq a_i \leq 50$ )。接下来有一系列命令，命令有4种形式：

- (1) Add  $i$   $j$ ,  $i$ 和 $j$ 为正整数,表示第 $i$ 个营地增加 $j$ 个人
  - (2) Sub  $i$   $j$ ,  $i$ 和 $j$ 为正整数,表示第 $i$ 个营地减少 $j$ 个人
  - (3) Query  $i$   $j$ ,  $i$ 和 $j$ 为正整数,  $i \leq j$ , 表示询问第 $i$ 到第 $j$ 个营地的总人数;
  - (4) End 表示结束，这条命令在每组数据最后出现;
- 问题分析：树状数组模板题，只是注意下命令读取，这种又有字符串，又有数字，还是老老实实用scanf吧！



# 树状数组应用

```
int main(){
    int temp,x,y;
    scanf("%d",&n);
    for(int i=1;i<=n;i++){
        scanf("%d",&temp);
        update(i,temp);
    }
    cin>>s;
    while(s!="End"){
        cout<<s<<endl;
        scanf("%d%d",&x,&y);
        if(s=="Query") printf("%lld\n",get(y)-get(x-1));
        if(s=="Add") update(x,y);
        if(s=="Sub") update(x,-y);
        cin>>s;
    }
    return 0;
}
```

## ➔ 逆序对

- **逆序对**：设有一个序列,对于序列中任意两个元素 $a_i, a_j$ , 若 $i < j$ 且 $a_i > a_j$ , 则说明 $a_i$ 和 $a_j$ 是一对逆序对。

给出一个长度为 $n$ 的序列 $A$ ,求出序列中逆序对的数量

- **数据范围**:

70%数据:  $n \leq 100000, a[i] \leq 100000$

100%数据:  $n \leq 100000, a[i] \leq 100000000$

求逆序对我们介绍三个方法:

- 1、暴力方法, 冒泡排序(自我实现)
- 2、70%数据, 普通树状数组
- 3、100%数据, 离散化树状数组或归并排序

## ➔ 树状数组求逆序对

- 问题分析：重温树状数组的思想：**从小到大的更新(向上)**，**从大到小的统计查询(向下)**。这与求逆序对有什么关系呢？我们思考这样子一个问题，如果我们已知一个元素的值为 $a[i]$ ，则与 $a[i]$ 构成逆序对的个数为，在 $i$ 元素后比 $a[i]$ 元素的个数。因此，我们以 $a[i]$ 的值建立树状数组的C数组，及 $C[a[i]]$ 。则对于一个 $a[i]$ ，我们只需要统计从 $i$ 到 $n$ 中比 $a[i]$ 值到1中出现的个数就行。所以我们可以对序列倒着枚举。**在来到 $a[i]$ 时，先向下统计，然后再插入 $a[i]$ ，即可。**
- 注意：这里使用 $a[i]$ 为C数组的下标，因此只能针对 $a[i]$ 值不太大的情况，如果太大，就不能直接用，需要离散化。

## → 树状数组求逆序对

```
void insert(int x){//从x位置
    while(x<=n){
        c[x]++;//仅统计个数
        x+=x&(-x);
    }
}

int get(int x){
    int ans=0;
    while(x>0){
        ans+=c[x];
        x-=x&(-x);
    }
    return ans;
}
```

```
int ans=0;
for(int i=n;i>=1;i--){
    ans+=get(a[i]-1);
    insert(a[i]);
}
```



## → 离散化

- 离散化，就是把一些很离散的点给重新分配。
- 举个例子，如果一个坐标轴很长( $> 1e10$ )，给你 $1e4$ 个坐标，询问某一个点，坐标比它小的点有多少。
- 很容易就知道，对于 $1e4$ 个点，我们不必把他们在坐标轴上的位置都表示出来，因为我们比较有多少比它小的话，只需要知道他们之间的相对大小就可以，而不是绝对大小，这，就需要离散化。
- **而离散化又分为两种，分为的两种是对于重复元素来划分的。第一种是重复元素离散化后的数字相同，第二种就是不同；根据情况而选择。**

# 离散化方法一

- 第一种其实就是用一个辅助的数组把你要离散的所有数据存下来。
- 然后对辅助数组排序，排序是为了后面的二分。
- 对辅助数组去重，因为我们要保证相同的元素离散化后数字相同。
- 再用二分把离散化后的数字放回原数组。

```
#include<algorithm> // 头文件
//n:原数组大小; num原数组中的元素;
//lsh:离散化的数组; cnt:离散化后的数组大小
int lsh[MAXN],cnt,num[MAXN],n;
for(int i=1; i<=n; i++) {
    scanf("%d",&num[i]);
    int __cdecl scanf (const char * __restrict __Format, ...)
}
sort(lsh+1,lsh+n+1);
cnt = unique(lsh+1,lsh+n+1)-lsh-1;
for(int i=1; i<=n; i++)
    num[i] = lower_bound(lsh+1,lsh+cnt+1,num[i])-lsh;
```

例如：输入序列为 1 100 9 7 6 6，离散化后为：1 5 4 3 2 2

## 离散化方法二

- 第二种方式其实就是排序之后，枚举着放回原数组
- 用一个结构体存下原数值及位置，然后按照原数值排序
- **最后离散化的下标大小关系=原数值的大小关系**，并可以把原数值的大小压缩成元素个数大小。
- 比如1 100 9 7这四个输，对应的是 $a[1]=1, a[2]=100, a[3]=9, a[4]=7$ ，按照值从小到大排序，则变成1 7 9 100，下标为1 4 3 2，则下标大小顺序与原值大小

```
struct Node {
    int data, id;
    bool operator < (const Node &a) const {
        return data < a.data;
    }
};
Node num[MAXN];
int rank[MAXN], n;
for(int i=1; i<=n; i++) {
    scanf("%d", &num[i].data); num[i].id = i;
}
sort(num+1, num+n+1);
for(int i=1; i<=n; i++) rank[num[i].id] = i;
```



# → 乒乓球比赛

## ➤ 问题描述

$n$ 个人按次序排列，每个人乒乓球技能值为 $a_i$ ，互不相同  
每场比赛需要两名选手，一名裁判，裁判的位置和技能值都必须在中间

问一共能组织多少场这样的比赛 ( $n \leq 20000$ )

## ➤ 问题分析

枚举裁判 $i$ ，计算 $i$ 之前有多少比 $a_i$ 小， $i$ 之后又有多少比 $a_i$ 大；以数值作为下标，开两个树状数组即可。



## ➔ 树状数组应用

### ➤ 问题2：区间更新，单点查询

- 从理论上来说，树状数组只能用于点修改，区间查询。如果用于区间修改，实际上是特别慢的。比如 $[1, n]$ 整个区间的数都 $+2$ 。这就需要把所有的 $a$ 的值都 $+2$ 。然后重新更新树状数组，效率远低于 $O(n)$ 。
- 但是在求区间之和的时候，却可以用于区间修改，点查询。但是实质是用单点修改代替区间修改，把单点查询变换为区间之和。
- 利用差分思想，区间 $[x, y]$ 都增加 $k$ ，实际上区间内所有的 $a[i+1]-a[i]$ 的值是不会发生任何改变的。改变的只有 $a[x+1]-a[x]$ 和 $a[y+1]-a[y]$ 。所以我们可以把区间修改变为单点修改。把点看成是差分之和。**(差分思想本来本来就是把区间更新，变成单点更新，前缀和统计的是单点跟新的值)**。所以在区间更新时，就是 $a[x]$ 向上加 $+\text{delta}$ ， $a[y+1]$ 位置向上 $-\text{delta}$ 的值。查询时：从 $x$ 位置查询到 $1+a[i]$ 的值即可。

# 树状数组应用

## ➤ 问题3：区间更新，区间查询

- 树状数组的区间修改区间查询稍微复杂一点，其实也比较简单，用习惯了就好，他主要还是基于差分的思想，我们分析一下：

**设原数组为 $a[i]$ ，差分数组为 $d[i]$ ，前缀和数组为 $c[i]$ 。那么区间更新我们就可以用 $d$ 数组来实现，关键统计 $c$ 数组可以用 $d$ 数组来维护吗？**

根据差分数组，我们知道：

$$a[1]=d[1]; a[2]=d[1]+d[2]; a[3]=d[1]+d[2]+d[3]; \dots$$

$$a[n]=d[1]+d[2]+\dots+d[n];$$

观察一下 $c$ 数组和 $d$ 数组的关系

$$c[x]=a[1]+a[2]+\dots+a[x]$$

$$=d[1]+(d[1]+d[2])+(d[1]+d[2]+d[3])+\dots+(d[1]+d[2]+\dots+d[x])$$

$$=x*d[1]+(x-1)*d[2]+\dots+1*d[x]$$

对于 $c[x]$ 来说， $d[1]$ 被加了 $x$ 次， $d[2]$ 被加了 $x-1$ 次， $d[x]$ 被加了1次。

思考，**如何把这个表达式变成 $c$ 数组与 $d$ 数组前缀和有关系？**



## → 树状数组应用

要把  $c[x] = x*d[1] + (x-1)*d[2] + \dots + 1*d[x]$ ，维护成与  $d$  数组前缀和有关系，则我们减去一个  $(x+1)*\sum_{i=1}^x d[i]$ 。则表达式变为：

$$\begin{aligned}
 c[x] &= x*d[1] + (x-1)*d[2] + \dots + 1*d[x] \\
 &= x*d[1] - (x+1)*d[1] + (x-1)*d[2] - (x+1)*d[2] + \dots + \\
 &\quad 1*d[x] - (x+1)*d[x] + (x+1)*\sum_{i=1}^x d[i] \\
 &= (x+1)*\sum_{i=1}^x d[i] - (1*d[1] + 2*d[2] + \dots + x*d[x]) \\
 &= (x+1)*\sum_{i=1}^x d[i] - \sum_{i=1}^x d[i] * i
 \end{aligned}$$

于是我们得到一个求  $C[x]$  (前缀和) 与  $d[x]$  前缀和有关系的表达式。因此，**我们只需要维护两个树状数组就可以了，一个维护差分数组  $d[i]$ ，另一个维护  $d[i]*i$  的值。**



## → 树状数组应用

问题3：区间更新，区间查询,参考代码

```
void insert(int x,int val){
    while(x<=n){
        c1[x]+=val; c2[x]+=x*val;
        x+=x&(-x);
    }
}

long long get(int x){
    long long ans=0;
    while(x>0){
        ans+=(x+1)*c1[x]-c2[x];
        x-=x&(-x);
    }
    return ans;
}
```





## ➔ 树状数组对比分析

- 树状数组相对于即将学习的线段树来说，虽然他们的时间复杂度都是 $n\log n$ ，但是很明显，树状数组的常数要小得多，效率也要快得多，只是我们刚开始使用的时候可能不太习惯，等习惯之后就懒得再去写复杂繁琐的线段树了。
- 但是树状数组也有他的劣势：
  - (1) 无法求解区间极值问题，如果当前修改的值就是区间极值，那么修改之后区间的原极值你就知道了。
  - (2) 树状数组可以用来做简单的区间修改区间查询的题，但是很多比较难的题目中，操作并不是简单的加减，查询的并不是和，这就导致了树状数组很难维护，而线段树就可以很暴力的维护。

## ➔ 树状数组的扩展—二维

- 树状数组也有他的优势，就可以能够很快的拓展到二维甚至多维。就拿二维树状数组来说，也就是单点修改，平面查询，可以很快的用二维树状数组解决，如果是二维线段树，不但实现很复杂，而且代码长度翻倍(树套树)。
- 而二维树状数组实现很简单，只需要增加一行代码。

```
void updata(int x,int y,int z)
{
    for(int i=x;i<=n;i+=i&(-i))
        //在一维枚举上加一维，三维一样
        for(int j=y;j<=n;j+=j&(-j))
            c[i][j]+=z;
}
```

```
long long query(int x,int y)
{
    long long sum=0;
    for(int i=x;i>0;i-=i&(-i))
        for(int j=y;j>0;j-=j&(-j))
            sum+=c[i][j];
    return sum;
}
```



## ➔ 推销员

### ➤ 问题描述

去一处死胡同推销， $n$ 家用户，第 $i$ 家用户离入口距离为 $s_i$ ，推销积累疲劳值 $a_i$ ，每走1距离也会积累疲劳值1，需要推销 $x$ （ $x$ 从1取到 $n$ ）家不走多余路，最多能积累多少的疲劳值( $n \leq 100000$ )。

### ➤ 问题分析

若采用 $O(n^2)$ 算法，每一次增加，要么在距离内、要么在距离外找

满分要采用数据结构维护

树状数组（一个维护前缀，只记录疲劳值，一个维护后缀，疲劳值加距离）



## ➔ 数星星

### ➤ 问题描述

$n$  ( $n \leq 10^5$ ) 颗星星有各自坐标  $(x, y)$  ( $0 \leq x, y \leq 10^5$ )  
对于每颗星星，求两个坐标均不大于它的星星个数？

### ➤ 问题分析

这是一道二维数点问题  
我们按横坐标把星星排序  
则把纵坐标放入树状数组中即可



# ➔ 魔兽争霸

## ➤ 问题描述

有 $n$ 个食尸鬼，初始生命值为 $a_i$ ，有三种操作：

第 $i$ 个食尸鬼被攻击，扣了 $d$ 的生命值

第 $i$ 个食尸鬼被死亡骑士发动死亡缠绕，恢复 $d$ 的生命值

询问生命值第 $k$ 多的食尸鬼有多少生命值？

## ➤ 问题分析

将食尸鬼的生命值当做下标，放入树状数组

二分答案求第 $k$ 大



# → Apple Tree

## ➤ 问题描述

有一棵 $n$ 个节点的苹果树，一开始每个节点上都结有一个苹果  
每次可以把一个点上的苹果摘下，或让一个没有苹果的节点长出一个苹果  
也可以询问以某节点为根的子树中有多少苹果（数据为10万数量级）

## ➤ 问题分析

树形结构无法用树状数组

我们可以dfs遍历得出dfs序

树就成了序列，一棵子树就成了一个区间

树状数组维护即可



## → 简单题

### ➤ 问题描述

一个01序列，每次让一个区间内的数翻转，询问第 $l$ 个数是多少

### ➤ 问题分析

我们依然采用差分，发现行不通

0与1的翻转让我们想起了异或

于是用异或运算来做差分

另一个方面理解，我们可以把翻转操作看成加1

最后只关心某个数的奇偶性





## ➔ 树状数组练习题目

- 洛谷 P3374 树状数组1
- 洛谷 P3368 树状数组2
- 洛谷 P3371 线段树1（分别用线段树、树状数组完成）
- 洛谷 P3373 线段树2
- POJ 1195 Mobile phone

The background features a central point from which several wide, colorful rays (in shades of blue, orange, red, and grey) radiate outwards. Faint binary code (0s and 1s) is scattered across the background, and a small line graph with two fluctuating lines is visible on the right side.

# Thank you!